Always-On CLI Tool Recommendations for Gemini CLI Workflows

This document consolidates the recommended always-on, non-MCP command-line tools that can improve Gemini CLI workflows. The emphasis is on words, search, research, text processing, document handling, navigation, and shell ergonomics, while still including the foundational developer-facing tools that were previously recommended.

The guiding idea is simple: install a strong local toolkit, then instruct Gemini CLI to prefer those tools through shell usage and `GEMINI.md` preferences. Modern command-line roundups repeatedly recommend tools such as `rg`, `fd`, `bat`, `fzf`, `zoxide`, `jq`, `yq`, `eza`, `delta`, `hyperfine`, `dust`, `duf`, `procs`, `glow`, and related text-oriented utilities as clearer, faster, and more ergonomic than older Unix defaults.[1][2][3]

How to read this document

Each tool entry includes:

- **Intended use case** — what problem it solves.
- **How you might use it** — practical Gemini CLI or human workflow examples.
- **Select one?** — whether multiple tools overlap and only one should usually be chosen.

A tool can be good even if it overlaps with another one. In those cases, this document calls out the likely “pick one” decision so the final stack stays lean.

Selection principles

A lean always-on stack should usually prioritize:

- Fast search and discovery.
- Strong text and document handling.
- Readable terminal output.
- Good support for Markdown, PDF, HTML, CSV, JSON, and YAML.
- Low-friction note capture and recall.
- Minimal duplication unless there is a clear difference in workflow or philosophy.

Tiering

- **A tier** — high-impact defaults for almost everyone.
- **B tier** — strong upgrades that cover common adjacent workflows.
- **C tier** — useful but more situational or preference-dependent.
- **D tier** — niche, specialized, or redundant unless a specific use case exists.

A tier

1. ripgrep (`rg`)

- **Intended use case:** Fast recursive text search across notes, research folders, and repositories.
- **How you might use it:** Search a vault for phrases, scan transcripts, find recurring claims in Markdown files, or have Gemini use `rg` instead of `grep` for nearly all local search work.
- **Select one?:** Yes. Usually choose `rg` over `grep`, `ag`, and `ack` as the primary default search tool.

2. fd

- **Intended use case:** Fast file discovery by name, extension, or partial pattern.
- **How you might use it:** Find all PDFs modified recently, all Markdown notes containing a date pattern in the filename, or all transcripts in a project tree.

- **Select one?:** Yes. Usually choose `fd` over `find` for interactive use.

3. bat

- **Intended use case:** Read files with syntax highlighting and line numbers.
- **How you might use it:** Preview Markdown, JSON, YAML, shell scripts, or config files before deciding what Gemini should transform or summarize.
- **Select one?:** Yes. Usually choose `bat` over `cat` for human-facing display.

4. eza

- **Intended use case:** Rich directory listing with tree view, metadata, and git awareness.
- **How you might use it:** Explore writing projects, research folders, or note archives with a clearer structure than `ls`.
- **Select one?:** Yes. Usually choose `eza` over raw `ls` for interactive exploration.

5. fzf

- **Intended use case:** Fuzzy selection of files, lines, commands, notes, and history.
- **How you might use it:** Pick a note from thousands, select a PDF from search results, choose a heading from a large document, or combine it with `rg` and `fd` for quick retrieval.
- **Select one?:** No. `fzf` complements many tools rather than replacing them.

6. zoxide

- **Intended use case:** Intelligent directory jumping based on usage history.
- **How you might use it:** Jump directly to your notes vault, research folder, school folder, or writing workspace with a short alias instead of repeated `cd` paths.
- **Select one?:** Usually yes. Choose `zoxide` over older directory jumpers like `fasd` unless you specifically prefer `fasd`.

7. jq

- **Intended use case:** Parse, query, and transform JSON.
- **How you might use it:** Inspect API output, filter tool results, reformat structured data from search or export pipelines, or hand Gemini cleaner JSON to reason over.
- **Select one?:** Yes for JSON. `jq` is the default choice.

8. yq

- **Intended use case:** Parse and transform YAML.
- **How you might use it:** Edit config files, examine frontmatter in Markdown, handle app or workflow configs, or inspect exported note metadata.
- **Select one?:** Yes for YAML. `yq` is the default choice.

9. delta

- **Intended use case:** Readable diffs and patches.
- **How you might use it:** Compare two drafts, inspect git changes, review a large Markdown rewrite, or make Gemini output diffs that are easier to inspect.
- **Select one?:** Usually yes. Choose `delta` over raw diff output.

10. glow

- **Intended use case:** Render Markdown beautifully in the terminal.
- **How you might use it:** Read a README, article draft, note, prompt library, or exported research document without opening another app.
- **Select one?:** No. Complements `bat`; use `glow` when rendered Markdown is preferable.

11. pandoc

- **Intended use case:** Convert documents between Markdown, DOCX, HTML, PDF, EPUB, and more.
- **How you might use it:** Convert drafts to DOCX, extract a Markdown-based writing workflow into PDF, or normalize source material into a format Gemini can process better.

- **Select one?:** Yes. This is usually the primary document conversion tool.[4][5][6]

12. poppler-utils

- **Intended use case:** Extract and manipulate PDF text and metadata.
- **How you might use it:** Use `pdftotext` to convert a paper into searchable text, `pdfinfo` to inspect metadata, or split and combine PDFs for research organization.
- **Select one?:** No. A toolkit rather than a single competing tool.[7][8][9]

13. ripgrep-all (`rga`)

- **Intended use case:** Search inside PDFs, Office docs, EPUBs, and archives, not just plain text.
- **How you might use it:** Search your entire research library for a term across PDFs, DOCX exports, and ebooks in one pass.
- **Select one?:** Usually yes if your workflow is document-heavy. Keep `rg` as the base tool and add `rga` for rich documents.

14. newsboat

- **Intended use case:** Terminal RSS and Atom feed reading.
- **How you might use it:** Track blogs, AI updates, cybersec feeds, policy news, writing communities, or addiction recovery resources without social media algorithms.[10][11]
- **Select one?:** Usually yes. Choose `newsboat` as the default feed reader.

15. nb

- **Intended use case:** Plain-text note-taking and local knowledge base management.
- **How you might use it:** Keep quick notes, clipped references, writing fragments, prompts, or research logs in a terminal-friendly system with tagging and search.
- **Select one?:** Yes. Pick one core note CLI among `nb`, `jrnl`, Joplin CLI, or simpler note tools.[12][13]

16. vale

- **Intended use case:** Prose linting and style enforcement.
- **How you might use it:** Check clarity, style consistency, documentation tone, and repeated wording in articles, notes, reports, or prompts.[14][15][16]
- **Select one?:** Usually yes as the main prose linter. Keep `proselint` or `alex` only if their specific checks matter.

17. xh

- **Intended use case:** Human-friendly HTTP requests.
- **How you might use it:** Query APIs for research, inspect web responses, test content endpoints, or fetch structured output more readably than with `curl`.
- **Select one?:** Usually yes. Choose `xh` over `curl` for interactive use.

18. VisiData

- **Intended use case:** Interactive exploration of CSV, TSV, JSON, and tabular data.
- **How you might use it:** Inspect survey exports, search results, scraped datasets, bibliographies, and text-analysis results without leaving the terminal.[17][18]
- **Select one?:** Usually yes. Pick `VisiData` as the main interactive table explorer.

19. qsv

- **Intended use case:** High-speed CSV wrangling and analysis.
- **How you might use it:** Clean large CSVs, compute frequencies, search columns, apply similarity/sentiment helpers, or prep text datasets for further review.[19][20][21]
- **Select one?:** Usually yes over `xsv` if you want the richer actively developed fork.

20. hyperfine

- **Intended use case:** Benchmark shell commands or text-processing pipelines.
- **How you might use it:** Compare `rg` vs `ag`, `qsv` vs `csvkit`, or measure which extraction command is fastest on large corpora.

- **Select one?:** Yes. Default benchmark tool.

B tier

21. dust

- **Intended use case:** Directory size inspection.
- **How you might use it:** Find which folders of PDFs, audio, datasets, or note attachments are consuming storage.
- **Select one?:** Usually yes. Prefer `dust` for interactive folder size analysis.

22. duf

- **Intended use case:** Filesystem usage overview.
- **How you might use it:** Quickly check disk usage across drives before downloading large corpora or media collections.
- **Select one?:** Yes for a friendlier `df` replacement.

23. procs

- **Intended use case:** Better process listing.
- **How you might use it:** Inspect resource-heavy processes while multiple AI tools, transcriptions, searches, or browsers are running.
- **Select one?:** Usually yes over raw `ps`.

24. btop

- **Intended use case:** Interactive system monitoring.
- **How you might use it:** Watch CPU, memory, network, and process activity while running batch OCR, large searches, or audio processing.
- **Select one?:** Usually choose one of `btop` or `htop`; `btop` is the nicer modern choice.

25. tokei

- **Intended use case:** Code statistics.
- **How you might use it:** Less important for your workflow, but still useful when a project mixes notes, scripts, and automation code.
- **Select one?:** Usually yes if you want code metrics; otherwise optional.

26. sd

- **Intended use case:** Simpler text substitution than `sed`.
- **How you might use it:** Rename repeated phrases across notes, normalize headers, or clean exported text without memorizing `sed` syntax.
- **Select one?:** Usually yes for interactive replacements.

27. ast-grep

- **Intended use case:** Structural code search and replacement.
- **How you might use it:** Mostly useful if you still do some code or automation refactoring.
- **Select one?:** Usually yes if structural code refactoring matters; otherwise skip.

28. just

- **Intended use case:** Repeatable command recipes.
- **How you might use it:** Create tasks like `just weekly-review`, `just export-notes`, `just search-library`, or `just build-doc` so Gemini or you can trigger repeatable workflows consistently.
- **Select one?:** Usually yes over a homemade shell script pile.

29. entr

- **Intended use case:** Re-run commands when files change.
- **How you might use it:** Auto-regenerate a document when notes change, auto-run `vale` on drafts, or re-export cleaned text after edits.
- **Select one?:** Yes for lightweight file-watch workflows.

30. tldr

- **Intended use case:** Practical command examples.
- **How you might use it:** Quickly recall usage patterns for obscure tools without digging into long man pages.
- **Select one?:** Yes. Great general companion.

31. csvlens

- **Intended use case:** Fast, focused CSV viewer.
- **How you might use it:** Open a CSV export for quick inspection before deciding whether deeper analysis is needed.
- **Select one?:** Usually choose between `csvlens` and `VisiData`; keep both only if you want a simple viewer plus a full explorer.[22][23]

32. csvkit

- **Intended use case:** CSV conversion and manipulation toolkit.
- **How you might use it:** Convert spreadsheets to CSV, run SQL-like inspection, or normalize tabular exports before analysis.
- **Select one?:** Usually choose between `csvkit`, `qsv`, and `xsv`. Keep one or two, not all three.

33. textql

- **Intended use case:** SQL queries over CSV/TSV and structured text.
- **How you might use it:** Query a text-heavy spreadsheet export with SQL syntax rather than awk-style pipelines.
- **Select one?:** Choose one among `textql`, `q`, and `csvq` depending on which interface feels best.[24]

34. q

- **Intended use case:** SQL directly on CSV/TSV files.
- **How you might use it:** Run joins or filters on exported data without loading a database.[25][26]
- **Select one?:** Same overlap group as `textql` and `csvq`; usually only one is needed.

35. csvq

- **Intended use case:** SQL-like querying for CSV and other delimited data.
- **How you might use it:** Explore text datasets with a familiar SQL approach while staying in a single CLI environment.[27]
- **Select one?:** Same overlap group as `textql` and `q`; one is usually enough.

36. docx2txt

- **Intended use case:** Extract plain text from DOCX files.
- **How you might use it:** Make a Word draft searchable with `rg`, compare versions, or feed cleaner text into another pipeline.[28][29]
- **Select one?:** Usually yes as the lightweight DOCX-to-text extractor.

37. w3m

- **Intended use case:** Text-mode web browsing and HTML-to-text conversion.
- **How you might use it:** Read web pages distraction-free, fetch readable HTML, or inspect content with less browser overhead.[30][31][32]
- **Select one?:** Choose one text browser among `w3m`, `lynx`, and `links` unless you like comparing them.

38. lynx

- **Intended use case:** Classic terminal web browsing.
- **How you might use it:** Quick low-bandwidth reading of articles or documentation from the shell.[33][34]
- **Select one?:** Same overlap group as `w3m` and `links`; one is usually enough.

39. links

- **Intended use case:** Terminal browser with a different rendering style and features.
- **How you might use it:** Alternate text browser when `w3m` or `lynx` behaves poorly on a site.[35][36]

- **Select one?:** Same overlap group as `w3m` and `lynx`; one is usually enough.

40. jrnl

- **Intended use case:** Fast command-line journaling.
- **How you might use it:** Keep recovery reflections, behavior logs, writing diary entries, and simple timestamped records.[37]
- **Select one?:** Choose between `jrnl`, `nb`, and Joplin CLI as the main note/journal tool.

41. Joplin CLI

- **Intended use case:** Terminal interface to a broader notes ecosystem.
- **How you might use it:** Manage Markdown notes with sync and encryption while keeping a text-first workflow.
- **Select one?:** Same overlap group as `nb` and `jrnl`; choose based on whether you want sync and app ecosystem support.

42. wordgrinder

- **Intended use case:** Distraction-free long-form writing in the terminal.
- **How you might use it:** Draft essays, articles, letters, or personal writing sessions with minimal UI clutter.[38][39]
- **Select one?:** Choose one terminal-centered writing environment among `wordgrinder`, `micro`, `vim`, or `helix` depending on comfort level.

43. micro

- **Intended use case:** Friendly terminal editor.
- **How you might use it:** Fast edits to Markdown, notes, prompts, or config files without needing modal editing habits.
- **Select one?:** Choose among `micro`, `vim`, `helix`, and `emacs` based on preference.

44. starship

- **Intended use case:** Information-rich shell prompt.
- **How you might use it:** See working directory, git state, environment, and context at a glance while switching among many writing and research folders.
- **Select one?:** Usually yes as the shell prompt layer.

45. direnv

- **Intended use case:** Per-directory environment variables.
- **How you might use it:** Auto-load project-specific API keys, aliases, or workspace settings when entering a folder.
- **Select one?:** Usually yes if you manage multiple project environments.

C tier

46. ack

- **Intended use case:** Search optimized for source trees and text projects.
- **How you might use it:** Alternate search tool if you prefer its defaults or output style.
- **Select one?:** Same overlap group as `rg` and `ag`; usually not needed if `rg` is installed.

47. ag (The Silver Searcher)

- **Intended use case:** Fast recursive search.
- **How you might use it:** Backup or preference alternative to `rg`.
- **Select one?:** Same overlap group as `rg` and `ack`; usually pick only one primary search tool.

48. pdfgrep

- **Intended use case:** Search plain text extracted from PDFs directly.
- **How you might use it:** Grep a single PDF or a small folder of papers when you do not need full `rga` support.

- **Select one?:** Can coexist with `rga`, but `rga` is usually the broader choice.

49. gron

- **Intended use case:** Flatten JSON for grepping and shell filters.
- **How you might use it:** Turn nested tool output into line-based data that can be searched or diffed.
- **Select one?:** Usually complementary to `jq`, not a replacement.

50. htmlq

- **Intended use case:** CSS-selector querying for HTML.
- **How you might use it:** Pull article content, titles, or metadata from saved HTML pages.
- **Select one?:** Usually yes if you do shell-based HTML scraping.

51. plocate

- **Intended use case:** Indexed file lookup.
- **How you might use it:** Instantly find files anywhere on disk when interactive discovery speed matters more than live filesystem truth.
- **Select one?:** Can complement `fd`; different model.

52. choose

- **Intended use case:** Quick field extraction from text streams.
- **How you might use it:** Pull selected columns from plain-text outputs without full `awk` complexity.
- **Select one?:** Usually optional.

53. angle-grinder

- **Intended use case:** Log-style text analysis.
- **How you might use it:** Analyze structured logs or timestamped records from automation runs or web requests.
- **Select one?:** Useful if you inspect logs often.

54. jc

- **Intended use case:** Convert command output to JSON.
- **How you might use it:** Make legacy shell commands easier for Gemini or `jq` to consume.
- **Select one?:** Usually yes if you like JSON-centric pipelines.

55. jo

- **Intended use case:** Build JSON payloads from shell arguments.
- **How you might use it:** Compose API bodies cleanly from shell scripts or one-liners.
- **Select one?:** Usually complementary to `jq` and `jc`.

56. jless

- **Intended use case:** Interactive JSON viewing.
- **How you might use it:** Browse large nested JSON results more comfortably than with raw output.
- **Select one?:** Choose one of `jless` or a text editor plus `jq` if you want to stay lean.

57. fq

- **Intended use case:** Query binary and structured formats.
- **How you might use it:** Inspect unfamiliar file formats without writing code.
- **Select one?:** Usually niche.

58. xsv

- **Intended use case:** Fast CSV processing.
- **How you might use it:** Similar role to `qsv` for CSV slicing and stats.
- **Select one?:** Usually pick `qsv` instead of `xsv` unless you specifically want the original tool.

59. FuzPad

- **Intended use case:** Simple fuzzy-searchable notes.
- **How you might use it:** Keep lightweight local notes without needing a larger PKM system.
- **Select one?:** Same overlap group as `nb`, `jrnl`, and other note CLIs; pick one core note manager.

60. Note CLI

- **Intended use case:** Terminal note app with highlighting and TUI interaction.
- **How you might use it:** Collect snippets and emphasize recurring keywords visually.
- **Select one?:** Same overlap group as `nb`, `jrnl`, and similar tools.

61. Helix

- **Intended use case:** Modern modal editor.
- **How you might use it:** Draft text or edit Markdown with more structure than `micro` and easier defaults than classic Vim.
- **Select one?:** Choose one main editor.

62. Vim

- **Intended use case:** Highly efficient modal editing.
- **How you might use it:** If already learned, it remains excellent for notes, prompts, and drafts.
- **Select one?:** Same main-editor overlap group.

63. Emacs

- **Intended use case:** Deeply extensible editing environment.
- **How you might use it:** If you want org-mode, journaling, planning, and writing in one environment.
- **Select one?:** Same main-editor overlap group.

64. Tilde

- **Intended use case:** Friendly console editor.
- **How you might use it:** Light note editing in constrained environments.
- **Select one?:** Usually niche if `micro` exists.

65. aspell

- **Intended use case:** Spell checking.
- **How you might use it:** Spell-check prose or notes in a terminal-only environment.[40]
- **Select one?:** Choose one spell checker among `aspell` and `ispell`; `aspell` is usually the better default.

66. ispell

- **Intended use case:** Legacy spell checking.
- **How you might use it:** Only if already embedded in an existing workflow.[40]
- **Select one?:** Usually choose `aspell` instead.

67. proselint

- **Intended use case:** English prose linting.
- **How you might use it:** Catch clichés, weasel words, or style issues in drafts.[41][42]
- **Select one?:** Often overlaps with `vale`; keep one or both if their rule sets differ meaningfully for you.

68. alex

- **Intended use case:** Inclusive language and wording review.
- **How you might use it:** Check wording for unintended bias or insensitive phrasing in published writing.[40]
- **Select one?:** Usually complementary to `vale`, but optional.

69. Aiksaurus

- **Intended use case:** Command-line thesaurus.
- **How you might use it:** Find alternative wording while drafting in the terminal.[43][44]
- **Select one?:** Usually yes if you want synonym lookup.

70. sttr

- **Intended use case:** String transformations.
- **How you might use it:** Convert case, encode/decode, hash, or quickly transform text snippets.[45][46]
- **Select one?:** Usually optional but handy.

71. XMLStarlet

- **Intended use case:** Query and transform XML.
- **How you might use it:** Work with XML corpora, exports, document formats, feeds, and metadata.[47][48][49]
- **Select one?:** Yes for XML-heavy work.

72. Asciidoctor

- **Intended use case:** Process AsciiDoc into publishable formats.
- **How you might use it:** If you encounter or adopt AsciiDoc instead of Markdown.[50][51]
- **Select one?:** Usually yes if you choose AsciiDoc.

73. AsciidoctorJ

- **Intended use case:** JVM-backed AsciiDoc processing.
- **How you might use it:** Alternative AsciiDoc runtime in Java-heavy environments.[52]
- **Select one?:** Usually choose either `asciidoctor` or `asciidoctorj`, not both.

74. taskwarrior

- **Intended use case:** CLI task management.
- **How you might use it:** Track article drafts, research tasks, sobriety routines, admin chores, or study tasks.

- **Select one?:** Choose one task manager if you want a dedicated CLI system.

75. taskwarrior-tui

- **Intended use case:** Interactive frontend for Taskwarrior.
- **How you might use it:** Review and triage tasks more comfortably.
- **Select one?:** Keep only if you already use Taskwarrior.

76. projectable

- **Intended use case:** Project management in terminal form.
- **How you might use it:** Group tasks and notes by project rather than keeping everything flat.
- **Select one?:** Usually optional.

77. nomino

- **Intended use case:** Bulk rename files.
- **How you might use it:** Normalize names of PDFs, lecture notes, scans, or exported assets.
- **Select one?:** Usually yes if you rename batches often.

78. neomutt

- **Intended use case:** Terminal email client.
- **How you might use it:** Process research newsletters, school mail, or administrative messages from the shell.
- **Select one?:** Usually only if email-in-terminal fits your lifestyle.

79. tut

- **Intended use case:** Mastodon client in terminal form.
- **How you might use it:** Track communities and news sources from a low-distraction environment.
- **Select one?:** Optional.

80. slack-term

- **Intended use case:** Slack from the terminal.
- **How you might use it:** Reduce context switching if you work with Slack-based groups or communities.
- **Select one?:** Optional.

D tier

81. elia

- **Intended use case:** Terminal client for LLMs.
- **How you might use it:** Secondary AI interface beside Gemini CLI.
- **Select one?:** Usually optional if Gemini CLI already covers the need.

82. hyphertool

- **Intended use case:** Hyphenation-aware wrapping.
- **How you might use it:** Improve line wrapping for long prose exports or terminal formatting.
- **Select one?:** Niche.

83. ssam

- **Intended use case:** Sampling text corpora.
- **How you might use it:** Create train/test/dev subsets from text datasets.
- **Select one?:** Mainly for NLP/data work.

84. lexmatch

- **Intended use case:** Lexicon matching in corpora.
- **How you might use it:** Search large document sets for vocabulary lists or dictionaries.
- **Select one?:** Niche.

85. stam

- **Intended use case:** Stand-off annotation.
- **How you might use it:** Attach annotations to texts without changing the source files.
- **Select one?:** Specialist use.

86. grex

- **Intended use case:** Generate regexes from example strings.
- **How you might use it:** Build regex patterns for filenames, citations, dates, or structured notes.
- **Select one?:** Optional, but useful if regex creation slows you down.

87. charfreq

- **Intended use case:** Character frequency analysis.
- **How you might use it:** Detect encoding weirdness or analyze corpora at a low level.
- **Select one?:** Niche.

88. lingua-cli

- **Intended use case:** Language detection.
- **How you might use it:** Detect mixed-language content in scraped corpora or multilingual notes.
- **Select one?:** Optional.

89. analiticcl

- **Intended use case:** Approximate string matching and OCR cleanup.
- **How you might use it:** Repair scanned text or normalize messy corpora.
- **Select one?:** Strong only for OCR-heavy workflows.

90. ucto

- **Intended use case:** Tokenization.
- **How you might use it:** Split text accurately before analysis.
- **Select one?:** Mostly NLP-specific.

91. frog

- **Intended use case:** NLP pipeline.
- **How you might use it:** More advanced linguistic analysis for suitable languages.
- **Select one?:** Specialist tool.

92. WordFlow

- **Intended use case:** Readable text-analysis summaries.
- **How you might use it:** Run word-level statistics over drafts or corpora.
- **Select one?:** Optional.

93. pbcopy / pbpaste

- **Intended use case:** Clipboard bridge on macOS.
- **How you might use it:** Move shell output into notes or drafts instantly.
- **Select one?:** Environment-specific.

94. xclip

- **Intended use case:** Clipboard bridge on Linux.
- **How you might use it:** Same role as `pbcopy/pbpaste`, but on X11 systems.
- **Select one?:** Environment-specific.

95. pv

- **Intended use case:** Show progress through pipes.
- **How you might use it:** Monitor large conversions or batch text transformations.
- **Select one?:** Usually optional but useful.

96. vidir

- **Intended use case:** Edit file lists in an editor to rename or organize them.
- **How you might use it:** Curate filenames for notes, PDFs, and exported assets in bulk.
- **Select one?:** Optional.

97. sponge

- **Intended use case:** Safe in-place pipeline writes.
- **How you might use it:** Rewrite a file using a pipeline without truncation races.
- **Select one?:** Optional but very useful in shell workflows.

98. ts

- **Intended use case:** Timestamp shell output.
- **How you might use it:** Keep dated logs of research runs or automation outputs.
- **Select one?:** Optional.

99. GNU parallel

- **Intended use case:** Parallelize many shell jobs.
- **How you might use it:** Run extraction, search, or conversion over many files at once.
- **Select one?:** Usually yes if you do frequent batch work.

100. gh

- **Intended use case:** GitHub CLI.
- **How you might use it:** Less central for your workflow, but useful for automation repos, issue handling, and documentation projects.
- **Select one?:** Optional if you use GitHub a lot.

101. dog

- **Intended use case:** Friendlier DNS queries.
- **How you might use it:** Useful when researching hosting or networking issues.
- **Select one?:** Optional.

102. fx

- **Intended use case:** Interactive JSON viewer.
- **How you might use it:** Explore nested JSON quickly without full jq expressions.
- **Select one?:** Overlaps with `jless`; pick one if needed.

103. hexyl

- **Intended use case:** Colorized hex dumps.
- **How you might use it:** Inspect binary files or weird encodings.
- **Select one?:** Niche.

104. lnav

- **Intended use case:** Log navigation and analysis.
- **How you might use it:** Browse logs from pipelines, scrapers, or local services.
- **Select one?:** Optional unless logs are central.

105. ncdu

- **Intended use case:** Interactive disk usage analysis.
- **How you might use it:** Alternate to `dust` for investigating large directories.
- **Select one?:** Choose between `ncdu` and `dust` for primary interactive disk analysis.

106. nnn

- **Intended use case:** Lightweight terminal file manager.
- **How you might use it:** Browse and manipulate files in a more navigational TUI form.
- **Select one?:** Pick one file manager among `nnn`, `xplr`, or `broot`.

107. xplr

- **Intended use case:** TUI file manager with plugins and layout flexibility.
- **How you might use it:** Structured browsing of large note or media collections.
- **Select one?:** Same overlap group as `nnn` and `broot`.

108. broot

- **Intended use case:** Tree navigation and searching.
- **How you might use it:** Explore big directory trees quickly.
- **Select one?:** Same overlap group as `nnn` and `xplr`.

109. zellij

- **Intended use case:** Modern terminal multiplexer.
- **How you might use it:** Keep persistent panes for Gemini CLI, notes, RSS, and search tasks.
- **Select one?:** Usually choose one of `zellij` or `tmux`.

110. tmux

- **Intended use case:** Traditional terminal multiplexer.
- **How you might use it:** Long-running sessions, pane splits, and remote persistence.
- **Select one?:** Usually choose one of `tmux` or `zellij`.

111. croc

- **Intended use case:** Secure file transfer.
- **How you might use it:** Send notes, exports, or reports across devices.
- **Select one?:** Optional.

112. magic-wormhole

- **Intended use case:** Human-friendly secure transfer.
- **How you might use it:** Share files quickly with a code phrase.
- **Select one?:** Usually choose one of `croc` or `magic-wormhole`.

113. tmate

- **Intended use case:** Shared terminal sessions.
- **How you might use it:** Remote collaboration or troubleshooting.
- **Select one?:** Optional.

114. thefuck

- **Intended use case:** Fix mistyped commands.
- **How you might use it:** Recover quickly from command mistakes in a dense CLI workflow.
- **Select one?:** Optional quality-of-life tool.

115. fasd

- **Intended use case:** Jump to recent/frequent files and directories.
- **How you might use it:** Alternative to `zoxide` if you prefer its behavior.
- **Select one?:** Usually pick `zoxide` or `fasd`, not both.

116. citeman

- **Intended use case:** CLI citation management.
- **How you might use it:** Build and maintain BibTeX references from the shell.[53]
- **Select one?:** Choose one citation helper if you need local bibliography management.

117. cites

- **Intended use case:** Fetch citations from DOI/CrossRef.
- **How you might use it:** Generate quick bib entries for papers from the terminal.[54]
- **Select one?:** Same overlap group as `citeman` depending on your citation flow.

118. viu

- **Intended use case:** View images in the terminal.
- **How you might use it:** Quick visual confirmation of screenshots, diagrams, or assets.
- **Select one?:** Optional.

119. gitui

- **Intended use case:** Terminal UI for git.
- **How you might use it:** Manage versioned notes or documentation repos more comfortably.
- **Select one?:** Choose one of `gitui` or `lazygit` if desired.

120. lazygit

- **Intended use case:** Another terminal UI for git.
- **How you might use it:** Stage, commit, and review changes to note or doc repositories.
- **Select one?:** Same overlap group as `gitui`; one is enough.

Best “pick one” overlap groups

To avoid installing five tools for the same job, these are the main groups where one primary choice is usually enough:

Purpose	Recommended primary choice	Alternatives
Recursive text search	rg	ag, ack
File discovery	fd	find, plocate
File display	bat	cat
Directory listing	eza	ls
Markdown reading	glow	bat
Directory jumping	zoxide	fasd
Prose linting	vale	proselint, alex
Notes/PKM	nb	jrnl, Joplin CLI, FuzPad, Note CLI
Main editor	micro or helix	vim, emacs, tilde, wordgrinder
CSV toolkit	qsv	csvkit, xsv
Interactive tabular viewer	VisiData	csvlens
SQL-on-CSV	q or textql	csvq
Text browser	w3m	lynx, links
Disk usage explorer	dust	ncdu
Terminal multiplexer	zellij or tmux	the other one
Git TUI	lazygit or gitui	the other one
Secure transfer	croc or magic-wormhole	the other one
Citation helper	citeman or cites	the other one

Recommended stack for a words/search/research-heavy workflow

If the goal is to keep the stack strong but not bloated, this is the shortlist most likely to matter day to day:

- rg
- rga
- fd
- bat
- eza
- fzf
- zoxide
- jq
- yq
- glow
- pandoc
- poppler-utils
- newsboat
- nb
- vale
- xh
- VisiData
- qsv
- just
- tldr
- micro or helix
- w3m
- Aiksaurus
- docx2txt
- dust
- duf
- btop

Example GEMINI.md preference block

```
## Tool preferences
- Use `rg` instead of grep for local text search.
- Use `rga` when searching PDFs, DOCX, EPUB, or archives.
- Use `fd` instead of find for file discovery.
- Use `bat` instead of cat for displaying files.
- Use `eza` for directory listings and tree views.
- Use `fzf` when selecting among many files or notes.
- Use `zoxide` for fast directory jumping.
- Use `jq` for JSON and `yq` for YAML.
- Use `glow` when rendering Markdown is more useful than raw output.
- Use `pandoc` for document conversion.
- Use `pdftotext` and related poppler tools for PDF extraction.
- Use `newsboat` for feed reading.
- Use `nb` for local note capture and retrieval.
- Use `vale` for prose linting.
- Use `xh` instead of curl for interactive HTTP requests.
- Use `qsv` or `VisiData` for CSV and tabular inspection.
- Use `tldr` first when recalling command syntax.
```

Install script

This install script targets Debian/Ubuntu plus Rust-based cargo installs. It is intentionally split into apt and cargo sections because many of the best modern CLI tools are distributed that way. Some package names vary by distro, and a few niche tools may need alternate package managers or manual install.

```
#!/usr/bin/env bash
set -euo pipefail

sudo apt-get update
sudo apt-get install -y \
  curl wget git build-essential pkg-config libssl-dev \
  ripgrep fd-find bat jq yq pandoc poppler-utils lynx w3m links \
```

```
  newsboat aspell ispell xmlstarlet taskwarrior neomutt tmux \
  xclip pv moreutils parallel python3-pip

if ! command -v cargo >/dev/null 2>&1; then
  curl https://sh.rustup.rs -sSf | sh -s -- -y
  source "$HOME/.cargo/env"
fi

cargo install \
  eza fzf zoxide du-dust duf procs bottom \
  hyperfine xh sd grex hexyl viu starship choose htmlq \
  jless qsv csvlens mdcat

pip install --user \
  visidata csvkit jrnl proselint

npm install -g \
  @mermaid-js/mermaid-cli alex

echo "Install complete."
echo "Note: some tools may require distro-specific package names or manua
echo "- bat may install as batcat on Debian/Ubuntu"
echo "- fd may install as fdfind on Debian/Ubuntu"
echo "- vale, nb, wordgrinder, rga, glow, micro, helix, lazygit, gitui, zellij, cro
```

Installation notes and caveats

- On Debian or Ubuntu, `bat` may be exposed as `batcat` and `fd` as `fdfind`; symlinks are often used to normalize names.
- Some tools are easiest to install from release binaries rather than distro repositories.
- A few tools in this document are intentionally niche; the script focuses on the highest-value broadly available set.
- If a fully exhaustive install script is required for every listed tool across one target OS, that is feasible but better done as a second pass tailored to a single operating system.

Suggested next pass for a fully exhaustive setup

A truly complete one-command installation for every tool in this document would be best produced in a second pass with one operating system chosen first, for example:

- Ubuntu 24.04
- Debian 12
- Arch Linux
- macOS with Homebrew

That approach would let the install section be exact instead of approximate for package names, symlinks, and release binaries.